

Configuración Inicial Herramientas de Desarrollo

1. Configurar cuenta de Github

a. Generar PAT (Personal Access Token)

- i. Ingresar a <https://github.com/settings/apps>
- ii. Seleccionar la opción Tokens (classic)
- iii. Seleccionar Generar Nuevo Token > Generar Nuevo Token (Clásico)
- iv. Cumplir la verificación vía email
- v. Crear un nuevo Token de Acceso Personal
- vi. Seleccionar sin expiración
 1. Nota: esto NO es recomendado, los PAT deben tener una vigencia. Se dejará sin expiración para temas de desarrollo de la asignatura.

vii. Seleccionar todos los Scopes

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

Personal Access Token Seminario Laravel

What's this token for?

Expiration

30 days (Jul 17, 2026)

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☐ repo	Full control of private repositories
☐ repostatus	Access commit status
☐ repo_deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo_invite	Access repository invitations
☐ security_events	Read and write security events
☒ workflow	Update GitHub Action workflows
☒ write_packages	Upload packages to GitHub Package Registry
☐ read_packages	Download packages from GitHub Package Registry
☒ delete_packages	Delete packages from GitHub Package Registry
☒ admin_org	Full control of orgs and teams, read and write org projects
☐ write_org	Read and write org and team membership, read and write org projects
☐ read_org	Read org and team membership, read org projects
☐ manage_runners_org	Manage org runners and runner groups
☒ admin_public_key	Full control of user public keys
☐ write_public_key	Write user public keys
☐ read_public_key	Read user public keys
☒ admin_repo_hook	Full control of repository hooks
☐ write_repo_hook	Write repository hooks
☐ read_repo_hook	Read repository hooks

viii. Dar clic en Generar Token

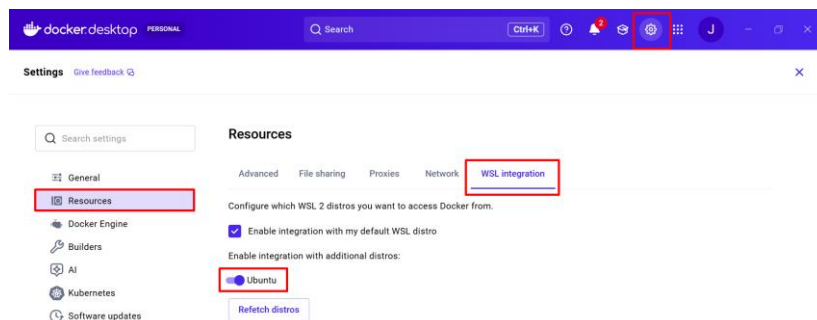
- ix. Almacenar el PAT generado en un lugar seguro debido a que no es posible cambiarlo una vez creado, si se olvida el PAT, se deberá eliminar y crear otro.
- x. A partir de ese punto, la contraseña será reemplazada por el PAT cada vez que se realice un Pull o un Push.

2. Verificar e instalar WSL (Windows Subsystem for Linux)

- a. Abrir PowerShell
- b. Ejecutar el commando: **wsl -l -v**
 - i. Es el acrónimo de:
 1. l: list
 2. v: verbose
- c. Debe mostrar: Ubuntu Running Version 2
- d. Luego, ejecutar el comando: **wsl --update**
- e. Posteriormente ejecutar: **wsl --install**
 - i. Esto instalará Ubuntu por defecto
 - ii. Si se quiere instalar otra distribución de Linux, se puede realizar de la siguiente forma:
 1. Validar la lista de distribuciones en línea: **wsl --list --online**
 - a. El repositorio oficial de imágenes está en <https://hub.docker.com/>
 2. Instalar la distribución deseada: **wsl --install -d Debian**
 - a. Cambiar “Debian” por la distribución que se requiera.

3. Instalar Docker Desktop

- a. Descargar desde la [página oficial](#).
- b. Configuración
 - i. En Docker Desktop: Settings → Resources → WSL Integration, habilitar: Ubuntu



- c. Validar Docker desde Ubuntu
 - i. Abrir Ubuntu

- ii. Ejecutar el comando: **docker -v**

```
jamescanos@CO-JCSANCHEZ:~$ docker --version
Docker version 29.4.2, build 055a478
```

- iii. Realizar prueba. Dentro de Ubuntu ejecutar:

1. **docker run hello-world**

```
jamescanos@CO-JCSANCHEZ:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760fdabc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

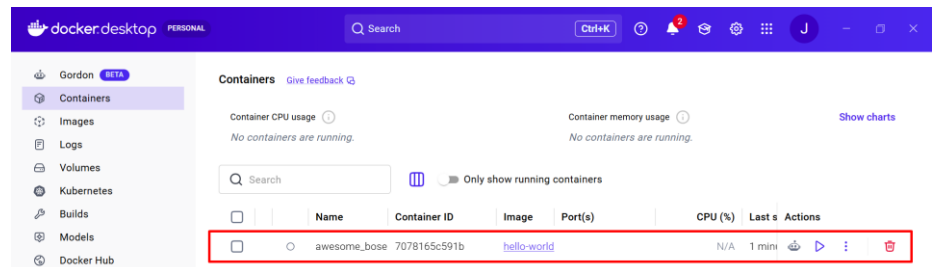
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

2. Validar en Docker Desktop que el contenedor está configurado correctamente:



- d. ¿Qué es Docker Compose?

- i. Permite gestionar con un solo comando:

1. múltiples contenedores
2. redes
3. volúmenes

- e. Estructura básica de un **docker-compose.yml**

- i. **services:** Es la sección principal. Cada elemento listado aquí representa un contenedor que se levantará.
- ii. **image o build:** Define la imagen que usará el contenedor. image descarga una imagen preexistente, mientras que build crea una imagen a partir de un Dockerfile.
- iii. **ports:** Expone los puertos hacia el exterior (del equipo anfitrión al contenedor).

- iv. **volumes:** Monta carpetas o archivos de tu equipo anfitrión dentro del contenedor para que los datos persistan, aunque el contenedor se apague.
- v. **environment:** Pasa variables de entorno necesarias para la configuración de la aplicación.
- vi. **depends_on:** Controla el orden de arranque de los contenedores.
- vii. **volumes:** Declara los volúmenes para mantener los datos seguros.
- viii. **networks:** Configura las redes para aislar los contenedores o permitir que se comuniquen entre ellos mediante nombres de host

f. Ejemplo:

```
version: '3.8' # Indica la versión de la sintaxis del archivo

services: # Define los contenedores que se van a ejecutar

  mi-servicio-1: # Nombre del contenedor/servicio

    image: nombre-de-la-imagen:tag # Imagen oficial o propia

    container_name: nombre_contenedor

    ports:

      - "8080:80" # Mapeo de puertos (host:contenedor)

    environment:

      - VARIABLE_ENTORNO=valor

    volumes:

      - ./ruta-local:/ruta-contenedor # Persistencia de datos

    networks:

      - mi-red

  mi-servicio-2:

    build: . # Indica que debe construir una imagen usando un Dockerfile local

    depends_on:

      - mi-servicio-1 # Asegura el orden de inicio de los contenedores

volumes: # Volúmenes para almacenar datos de forma persistente

  mi-volumen:

networks: # Redes personalizadas para la comunicación entre contenedores

  mi-red:

    driver: bridge
```

g. Dockerfile: es un archivo para construir imágenes personalizadas. Se utilizan los siguientes comandos:

- i. **FROM:** Define la imagen base desde la cual empezar (por ejemplo, una versión oficial de Python, Node.js o Ubuntu).
- ii. **WORKDIR:** Establece el directorio de trabajo dentro del contenedor.
- iii. **COPY:** Copia archivos desde tu computadora al interior de la imagen de Docker.
- iv. **RUN:** Ejecuta comandos para instalar paquetes y dependencias (como `pip install` o `npm install`).
- v. **EXPOSE:** Indica el puerto de red en el que tu aplicación estará escuchando.
- vi. **CMD o ENTRYPOINT:** Define el comando que se ejecutará automáticamente cuando el contenedor arranque.

```
FROM php:8.2-apache

RUN docker-php-ext-install ldap

COPY . /var/www/html/
```

- h. `docker-compose.override`: sobrescribe la configuración inicial de manera local.
- i. Ejecutar contenedores: desde WSL
 - i. `docker compose up -d`
- j. Validar contenedores:
 - i. Desde Ubuntu ejecutar el siguiente comando: `docker ps`
- k. Para ejecutar los contenedores se realiza el siguiente comando: `docker compose up -d`

ACTIVIDAD GUIADA

Requisitos previos

1. **Docker Desktop** (con el Backend de WSL2 activado si usas Windows).
2. **Git** (para mantener los cambios actualizados en el repositorio).
3. **Visual Studio Code** (recomendado) con la extensión "Remote - Containers" o "Dev Containers" (opcional).

Estructura de archivos para el repositorio

Crear en Ubuntu, la estructura de directorios como se muestra a continuación:

|—BDII/

|— Clase II

|—docker-compose.yml

|— Dockerfile

|— src/ (carpeta vacía, solo se usa para código PHP)

|— README.md

Comandos para crear estructura:

```
mkdir BDII
```

```
cd BDII
```

```
mkdir ClaseII
```

```
cd ClaseII
```

```
touch docker-compose.yml
```

```
touch Dockerfile
```

```
mkdir src
```

```
touch README.md
```

Modificar Dockerfile

Ejecutar el comando **nano Dockerfile** y agregar el siguiente código:

```
# Se usa una imagen oficial de PHP con Apache incluido
FROM php:8.2-apache

# Se actualiza el sistema y se instalan las librerías necesarias para extensiones PHP
RUN apt-get update && apt-get install -y \
    libpng-dev \
    libjpeg-dev \
    libfreetype6-dev \
    zip \
    unzip \
    git \
    && docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install gd pdo pdo_mysql mysqli \
    && a2enmod rewrite

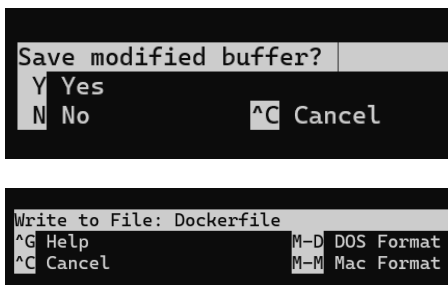
# Se habilita el módulo de reescritura de Apache (útil para Laravel después)
RUN a2enmod rewrite

# Se define el directorio de trabajo dentro del contenedor
WORKDIR /var/www/html

# Se copia un archivo de prueba por si la carpeta 'src' está vacía
RUN echo "<?php phpinfo(); ?>" > /var/www/html/index.php

# Se asignan permisos al usuario de Apache (www-data)
RUN chown -R www-data:www-data /var/www/html
```

Para guardar, presionar CTRL + X y luego colocar la letra “y”, posteriormente, presionar la tecla ENTER, para guardar el contenido del archivo.



Modificar docker-compose

Ejecutar el comando **nano docker-compose.yml** y agregar el siguiente código:

```
services:
  # Servicio de la aplicación PHP
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: bases_de_datos
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html
    networks:
      - bdd_network
    depends_on:
      - db

  # Servicio de base de datos (MariaDB/MySQL)
  db:
    image: mariadb:10.8
    container_name: bdd_mariadb
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: bdd_db
      MYSQL_USER: dev_user
      MYSQL_PASSWORD: dev_password
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - bdd_network

  # Servicio de phpMyAdmin (interfaz gráfica para la BD)
  phpmyadmin:
    image: phpmyadmin/phpmyadmin:latest
    container_name: bdd_phpmyadmin
    restart: unless-stopped
    environment:
      PMA_HOST: db
      PMA_USER: root
      PMA_PASSWORD: root_password
      UPLOAD_LIMIT: 100M
    ports:
      - "8081:80"
    networks:
      - bdd_network
    depends_on:
      - db

# Volúmenes persistentes (los datos de la BD persisten, aunque se borren los contenedores)
volumes:
  db_data:

# Red interna personalizada para que los contenedores se "vean" por nombre
networks:
  bdd_network:
    driver: bridge
```


Modificar README.md

Ejecutar el comando **nano README.md** y agregar el siguiente código:

```
# Entorno de Desarrollo - Bases de Datos II

Este repositorio contiene la configuración base del entorno de desarrollo usando **Docker**.
Con solo unos comandos se tendrá un servidor web (PHP 8.2), una base de datos (MariaDB) y phpMyAdmin
funcionando en la máquina local, sin necesidad de instalar XAMPP ni WAMP.

---

## Requisitos previos

1. **Docker Desktop** (con el backend de WSL2 activado si usa Windows).
   [Descargar Docker Desktop](https://www.docker.com/products/docker-desktop/)
2. **Git** (para clonar el repositorio).
3. **Visual Studio Code** (recomendado) con la extensión "Remote - Containers" o "Dev Containers"
   (opcional).

---

## Paso a paso para levantar el entorno

### 1. Clonar el repositorio
Abrir la terminal (WSL2 / PowerShell / Bash) y ejecuta:
```bash
git clone https://github.com/USUARIO/BDII.git
cd BDII
```

### 2. Estructura Inicial
Dentro de la carpeta, crea una carpeta llamada src
```bash
mkdir src
```

### 3. Levantar los contenedores
Ejecutar el siguiente comando en la raíz del proyecto (donde está el docker-compose.yml):
```bash
docker-compose up -d
```

El flag -d significa "detached" (corre en segundo plano). Si se desean ver los logs en vivo, se
quita el -d.

### 4. Verificar que todo funciona
PHP/Apache: Abrir el navegador y acceder a http://localhost:8080.
Se debe ver página de información de PHP (phpinfo()).

phpMyAdmin: Acceder a http://localhost:8081.
Usuario: root, Contraseña: root_password.

Base de datos: conectarse desde phpMyAdmin o desde su código PHP usando:
Host: db (el nombre del servicio en el compose)
Usuario: root (o dev_user)
Contraseña: root_password (o dev_password)
Base de datos: bdd_db
```

Levantar los contenedores

Ejecutar el siguiente comando en la raíz del proyecto (donde está el docker-compose.yml):

```
docker-compose up -d
```

El flag -d significa "detached" (corre en segundo plano). Si se desean ver los logs en vivo, se quita el -d.

Crear archivo de prueba

Ingresar al directorio src con el comando: *cd src*

Luego crear un archivo PHP de prueba: *nano index.php*

Colocar el siguiente código:

```
<?php  
    phpinfo();  
?>
```

Para guardar, presionar CTRL + X y luego colocar la letra "y", posteriormente, presionar la tecla ENTER, para guardar el contenido del archivo.

Verificar que todo funciona

PHP/Apache: Abrir el navegador y acceder a <http://localhost:8080>

Se debe ver página de información de PHP (phpinfo()).

phpMyAdmin: Acceder a <http://localhost:8081>.

Usuario: root, Contraseña: root_password.

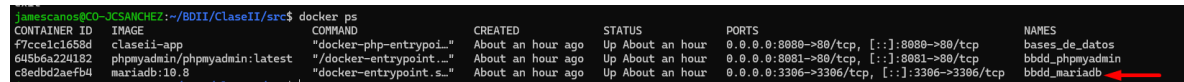
Base de datos: conectarse desde phpMyAdmin o desde su código PHP usando:

```
Host: db (el nombre del servicio en el compose)  
Usuario: root (o dev_user)  
Contraseña: root_password (o dev_password)  
Base de datos: bbdd_db
```

Acceder a contenedor de base de datos

Para acceder a un contenedor, lo primero que se debe validar es el nombre de este, para esto, se ejecuta el comando:

```
docker ps
```



| CONTAINER ID | IMAGE | COMMAND | CREATED | STATUS | PORTS | NAMES |
|--------------|------------------------------|--------------------------|-------------------|------------------|---|-----------------|
| f7ccelc1658d | claseii-app | "docker-php-entrypoi..." | About an hour ago | Up About an hour | 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp | bases_de_datos |
| 6d5b6a22d192 | phpmyadmin/phpmyadmin:latest | "/docker-entrypoint..." | About an hour ago | Up About an hour | 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp | bbdd_phpmyadmin |
| c8edbd2aefb4 | mariadb:10.8 | "docker-entrypoint.s..." | About an hour ago | Up About an hour | 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp | bbdd_mariadb |

En la columna NAMES, se muestran los nombres de todos los contenedores que se encuentran corriendo, para el caso de MariaDB, se puede conocer por el valor del puerto (PORTS) que generalmente es 3306, 3307 o 3308.

Una vez que se conoce el nombre del contenedor, se ejecuta el comando:

```
docker exec -it bbdd_mariadb bash
```

Cuando se accede, se puede acceder al contenedor de Linux del servicio creado y desde allí se puede acceder al motor de MariaBD con el siguiente comando:

```
mysql -u root -p
```

A su vez, solicitará la contraseña del usuario root, esta es la que se definió en el archivo docker-compose.yml y es "root_password"

```
root@c8edbd2aefb4:/# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 73
Server version: 10.8.8-MariaDB-1:10.8.8+maria~ubu2204 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> |
```

Con la consola activa, se pueden usar todos los comandos de SQL disponibles.

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| bbdd_db   |
| information_schema |
| mysql     |
| performance_schema |
| prueba_procedimientos |
| sys       |
+-----+
6 rows in set (0.001 sec)
```

Solución de problemas comunes

1. Puerto 8080 o 8081 ocupados:

Cambiar el puerto de la izquierda en el docker-compose.yml (ej: "8082:80").

2. phpMyAdmin no conecta con la BD:

Verificar que el servicio db esté levantado con docker-compose ps. Asegurarse de que la variable PMA_HOST sea exactamente db (el nombre del servicio).

3. Error de permisos al guardar archivos desde el contenedor:

En la terminal WSL, ejecutar: sudo chown -R \$USER:\$USER ./src para darle permisos del usuario a la carpeta local.

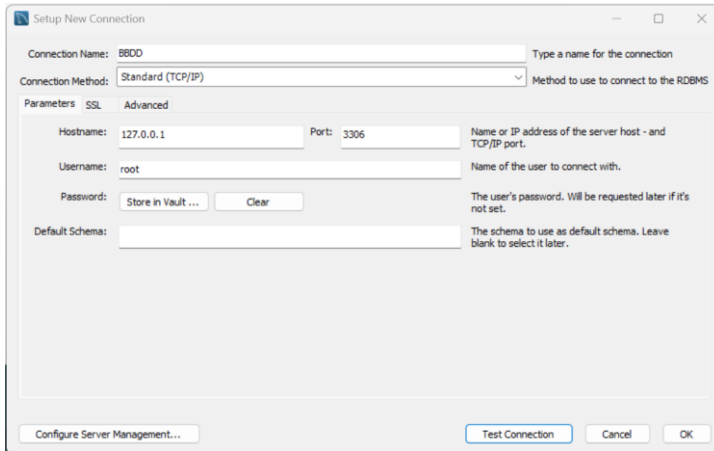
4. La carpeta src está vacía y no se ve el phpinfo():

Reconstruir la imagen: docker-compose up -d --build.

Conectar BBDD desde MySQL Workbench

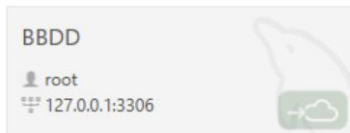
Seleccionar la opción Database > Connect to Database

Colocar un nombre para la conexión, Hostname, Puerto, Usuario y clave, luego, dar clic en el botón “OK”.



Luego aparecerá en las conexiones de MySQL en el inicio de MySQL Workbench:

MySQL Connections + ⓘ



Entregable de esta actividad

1. Suba este repositorio a su cuenta de GitHub (público, agregando al docente como colaborador).
2. Toma una captura de pantalla donde se vea:
 - La terminal mostrando docker-compose up -d exitoso.
 - El navegador en <http://localhost:8080> mostrando el phpinfo().
 - El navegador en <http://localhost:8081> mostrando el login de phpMyAdmin.

Procedimientos Almacenados

Es una porción de código que se puede guardar y reutilizar. Es útil cuando se debe repetir las mismas tareas en muchas ocasiones permitiendo la encapsulación de código.

Los procedimientos aceptan datos como parámetros de modo que actúa en función de estos.

Sintaxis de un Procedimiento

```
DELIMITER $$
CREATE PROCEDURE nombre_procedimiento
BEGIN
    AS
    nombre_sentencia
END$$;
```

¿Qué significa cada parte?

| Elemento | Explicación |
|-----------------------------|--|
| DELIMITER \$\$ | Cambia temporalmente el delimitador de las instrucciones. Permite utilizar ; dentro del procedimiento. |
| CREATE PROCEDURE | Indica que se va a crear un procedimiento almacenado. |
| nombre_procedimiento | Es el nombre que se asigna al procedimiento. Debe ser descriptivo. |
| () | Indica los parámetros que recibe el procedimiento. Puede estar vacío si no recibe parámetros. |
| BEGIN | Indica el inicio del bloque de instrucciones que ejecutará el procedimiento. |
| nombre_sentencia; | Representa una o varias instrucciones SQL que realizará el procedimiento. |
| END | Indica el final del bloque de instrucciones. |
| \$\$ | Indica el final de la creación del procedimiento. |
| DELIMITER ; | Restablece el delimitador original de MySQL. |

Ejemplo sencillo

```
DELIMITER $$  
CREATE PROCEDURE listarProductos()  
BEGIN  
SELECT * FROM producto;  
END$$  
DELIMITER ;
```

Ejecutar un procedimiento

Para ejecutar un procedimiento solo es necesario:

```
EXEC NOMBRE_PROCEDIMIENTO(parametro1,parametro2,...)
```

```
CALL listarProductos();
```

Dentro de un procedimiento se pueden utilizar variables que se deben definir con el símbolo arroba @

@variable

Procedimientos con parámetros

Los procedimientos reciben diferentes tipos de parámetros:

- Entrada (IN)
 - Es el tipo de parámetro que se usa por defecto. La aplicación o código que invoque al procedimiento tendrá que pasar un argumento para este parámetro. El procedimiento trabajará con una copia de su valor, teniendo el parámetro su valor original al terminar la ejecución del procedimiento.
- Salida (OUT)
 - El valor de este parámetro puede ser cambiado en el procedimiento, y además su valor modificado será enviado de vuelta al código o programa que invoca el procedimiento.
- Entrada/Salida (INOUT)
 - Es una mezcla de los dos conceptos anteriores. La aplicación o código que invoca al procedimiento puede pasarle un valor a éste, devolviendo el valor modificado al terminar la ejecución.

Cuando el procedimiento necesita recibir información, los parámetros se colocan dentro de los paréntesis:

```
DELIMITER $$

CREATE PROCEDURE obtenerProductoEstado(IN p_estado VARCHAR(20))
BEGIN
    SELECT *
    FROM producto
    WHERE estado = p_estado;
END$$

DELIMITER ;
```

En este caso:

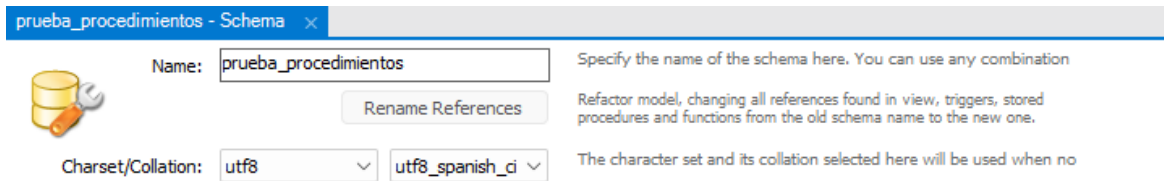
- IN indica que el procedimiento **recibe un dato**.
- p_estado es el nombre del parámetro.
- VARCHAR(20) indica el tipo de dato que recibe.
- WHERE estado = p_estado utiliza ese valor para realizar la consulta.

Ventajas y desventajas

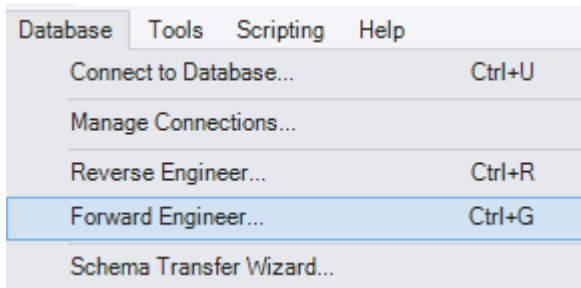
- Al reducir la carga en las capas superiores, se reduce el tráfico de red y, si el uso es el correcto, puede mejorar el rendimiento.
- Es posible limitar los permisos de acceso de usuario a los procedimientos almacenados y no a las tablas directamente.
- No todos los sistemas gestores de bases de datos usan los procedimientos del mismo modo, por lo que se reduce la portabilidad del código.
- Requiere una curva de aprendizaje para su implementación.

Ejemplo

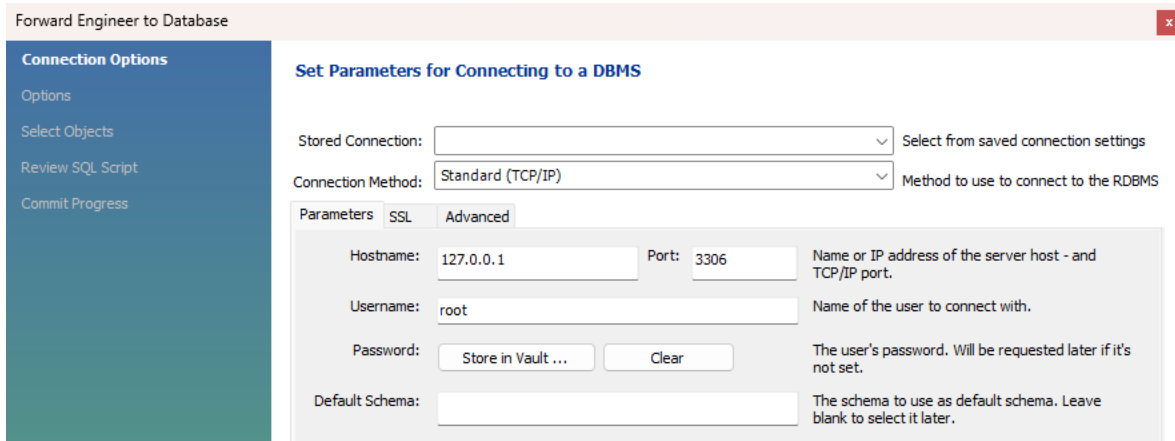
- Crear un esquema llamado prueba_procedimientos
- Seleccionar el esquema anteriormente creado.
- Crear una tabla llamada productos



Guardar el archivo como prueba_procedimientos.mwb, luego exportar para crear la base de datos, seleccionar la opción Database > Forward Engineer



Agregar los datos de la conexión donde se creará el esquema



Dejar marcadas las opciones seleccionadas por defecto:

Forward Engineer to Database

Connection Options

Options

Select Objects

Review SQL Script

Commit Progress

Set Options for Database to be Created

Tables

- ☐ Skip creation of FOREIGN KEYS
- ☐ Skip creation of FK Indexes as well
- ☐ Generate separate CREATE INDEX statements
- ☐ Generate INSERT statements for tables
- ☐ Disable FK checks for INSERTs

Other Objects

- ☐ Don't create view placeholder tables
- ☐ Do not create users. Only create privileges (GRANTS)

Code Generation

- ☐ DROP objects before each CREATE object
- ☐ Generate DROP SCHEMA
- ☐ Omit schema qualifier in object names
- ☐ Generate USE statements
- ☐ Add SHOW WARNINGS after every DDL statement
- ☒ Include model attached scripts

Forward Engineer to Database

Connection Options

Options

Select Objects

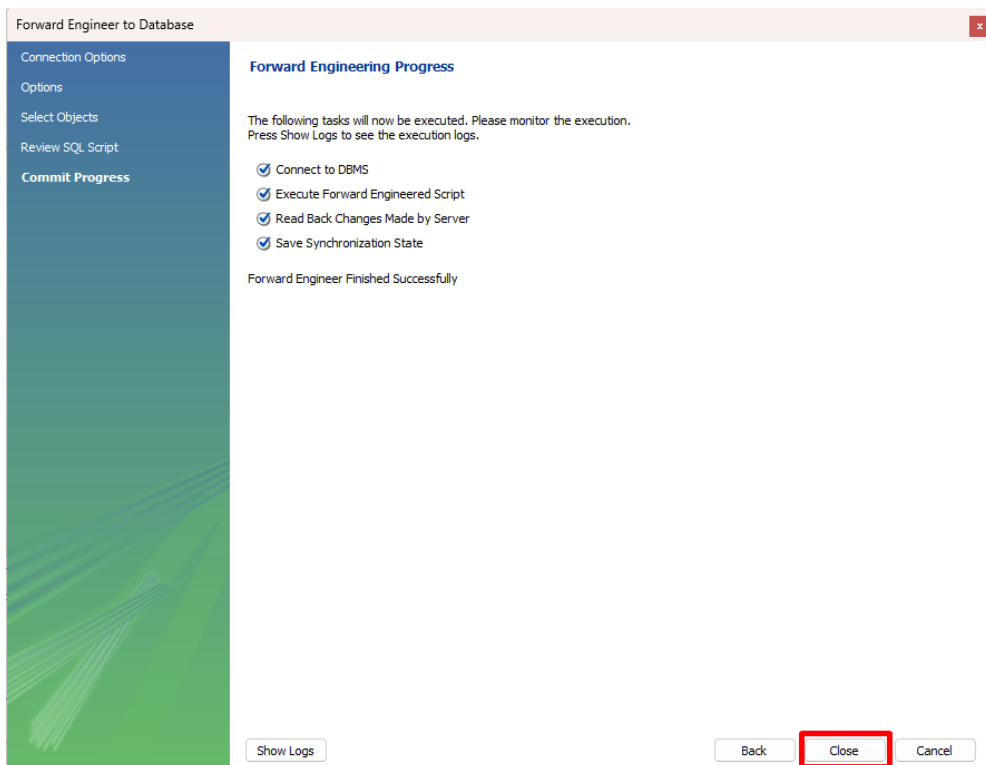
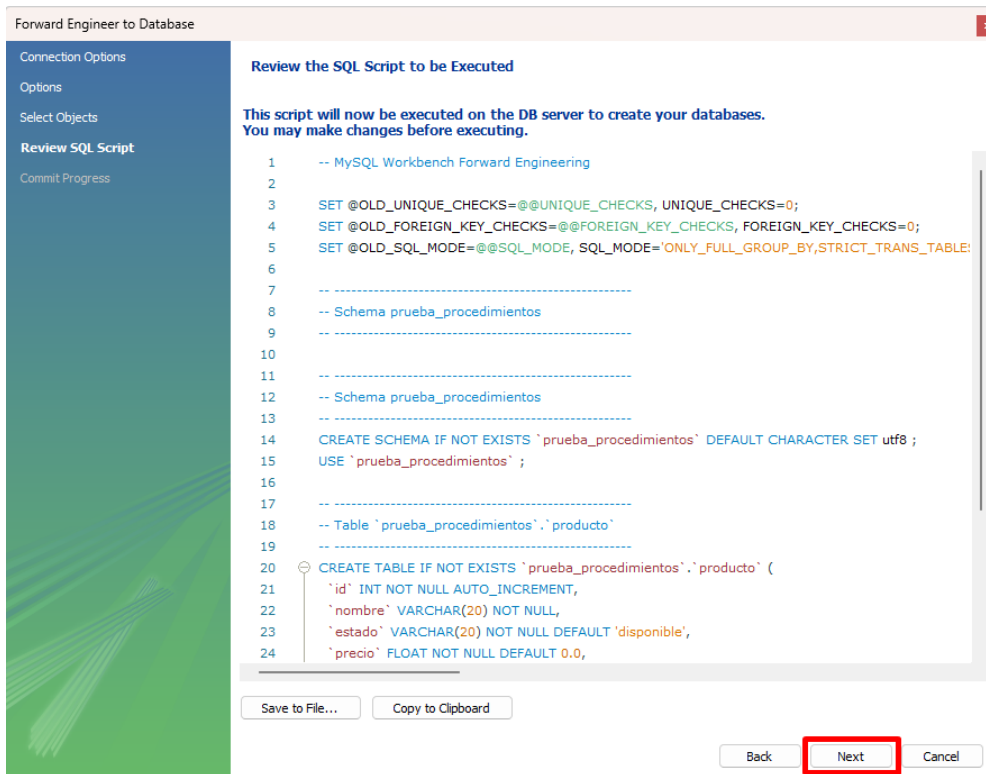
Review SQL Script

Commit Progress

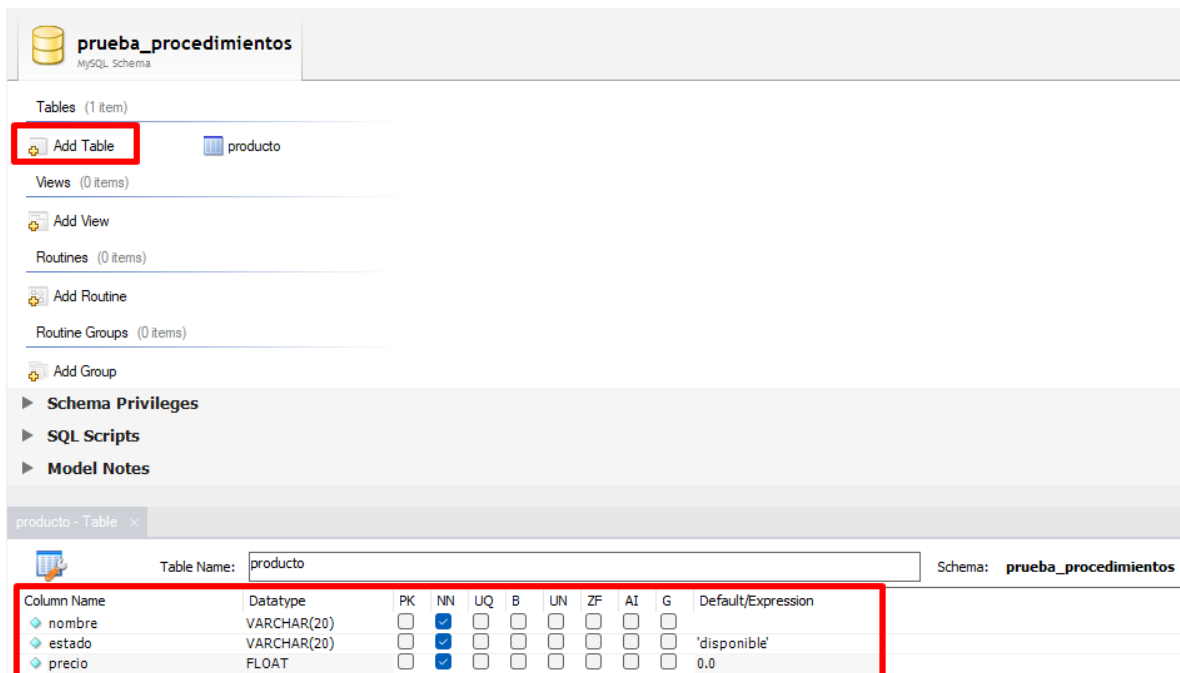
Select Objects to Forward Engineer

To exclude objects of a specific type from the SQL Export, disable the corresponding checkbox. Press Show Filter and add objects or patterns to the ignore list to exclude them from the export.

| | | | |
|---|--|-----------------------------|-------------|
|  | ☒ Export MySQL Table Objects | 1 Total Objects, 1 Selected | Show Filter |
|  | ☐ Export MySQL View Objects | 0 Total Objects, 0 Selected | Show Filter |
|  | ☐ Export MySQL Routine Objects | 0 Total Objects, 0 Selected | Show Filter |
| | ☐ Export MySQL Trigger Objects | 0 Total Objects, 0 Selected | Show Filter |
| | ☐ Export User Objects | 0 Total Objects, 0 Selected | Show Filter |



Crear Tabla producto



MySQL Schema: prueba_procedimientos

Tables (1 item)

Add Table producto

Views (0 items)

Add View

Routines (0 items)

Add Routine

Routine Groups (0 items)

Add Group

Schema Privileges

SQL Scripts

Model Notes

producto - Table

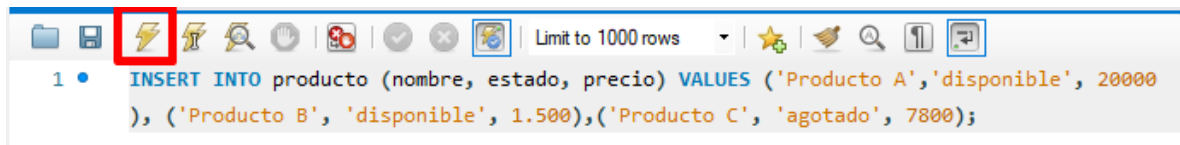
Table Name: producto Schema: prueba_procedimientos

| Column Name | Datatype | PK | NN | UQ | B | UN | ZF | AI | G | Default/Expression |
|-------------|-------------|--------------------------|-------------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------|
| nombre | VARCHAR(20) | ☐ | ☒ | ☐ | ☐ | ☐ | ☐ | ☐ | ☐ | |
| estado | VARCHAR(20) | ☐ | ☒ | ☐ | ☐ | ☐ | ☐ | ☐ | ☐ | 'disponible' |
| precio | FLOAT | ☐ | ☒ | ☐ | ☐ | ☐ | ☐ | ☐ | ☐ | 0.0 |

Insertar Registros

Ejecutar el siguiente comando para insertar registros en la tabla recién creada:

```
INSERT INTO producto(nombre, estado, precio) VALUES ('Producto A','disponible', 20000), ('Producto B', 'disponible', 1.500), ('Producto C', 'agotado', 7800);
```



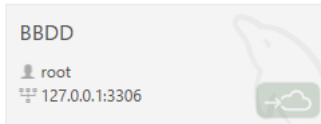
Limit to 1000 rows

```
1 • INSERT INTO producto (nombre, estado, precio) VALUES ('Producto A','disponible', 20000), ('Producto B', 'disponible', 1.500), ('Producto C', 'agotado', 7800);
```

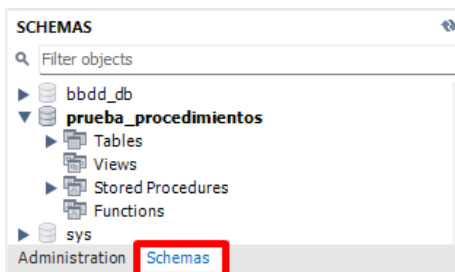
Crear procedimiento de Entrada

En el inicio de MySQL Workbench, seleccionar la conexión activa a la BBDD creada, dar doble clic:

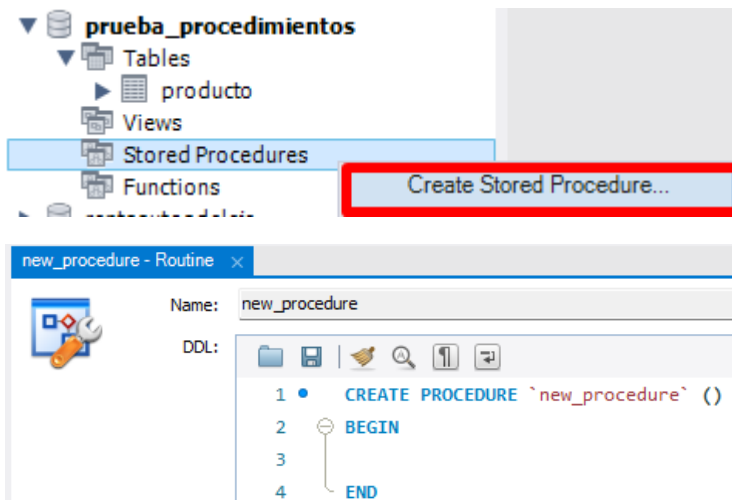
MySQL Connections ⊕ ⊖



Seleccionar la opción de esquemas “Schemas”:



Dar clic derecho sobre “Stored Procedures” y después “Create Stored Procedures”

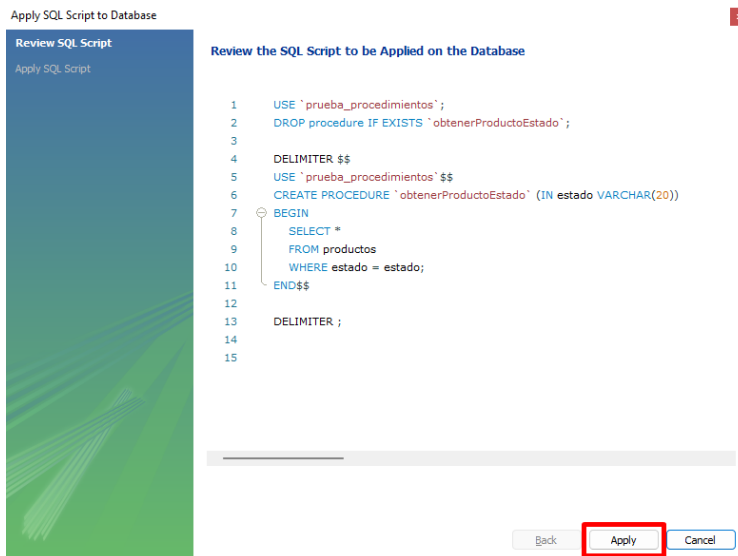


Crear el siguiente código:

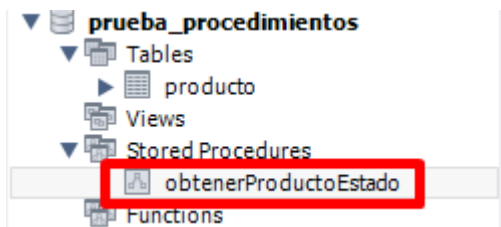
```
CREATE DEFINER=`root`@`%` PROCEDURE `obtenerProductoEstado` (
  IN p_estado VARCHAR(20)
)
BEGIN
  SELECT *
  FROM producto
  WHERE estado = p_estado;
END
```

Guardar Procedimiento

Una vez digitado el código, se procede a almacenar el procedimiento:

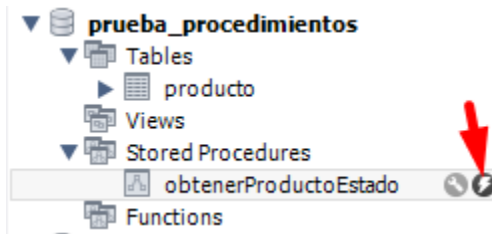


En la opción “Stored Procedures” se debe visualizar el procedimiento creado.

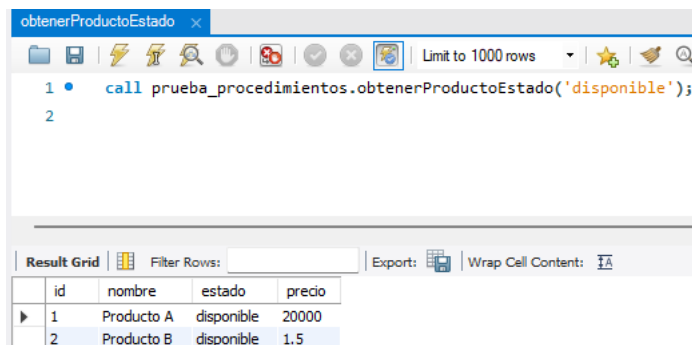
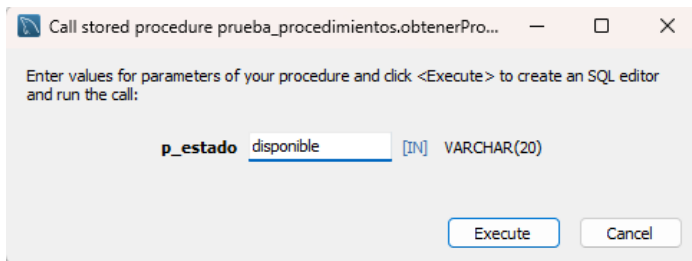


Ejecutar Procedimiento

Se puede ejecutar un procedimiento desde el panel a la izquierda en Workbench



Incluir el parámetro

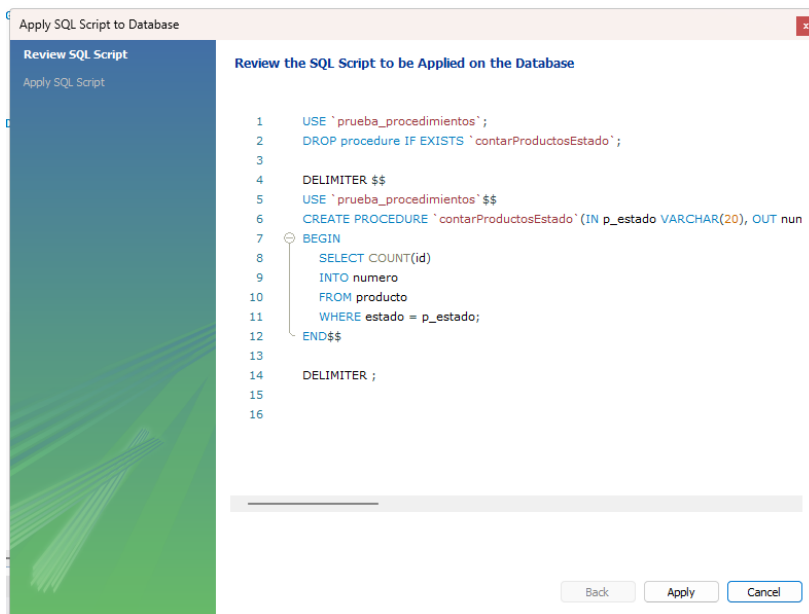
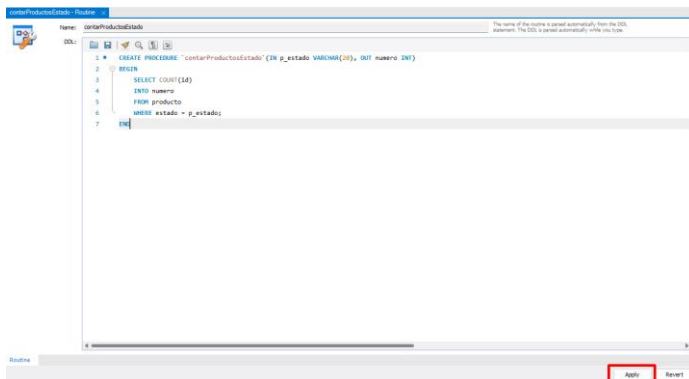


Procedimiento de Salida

Crear el procedimiento de salida contarProductosEstado con el siguiente código:

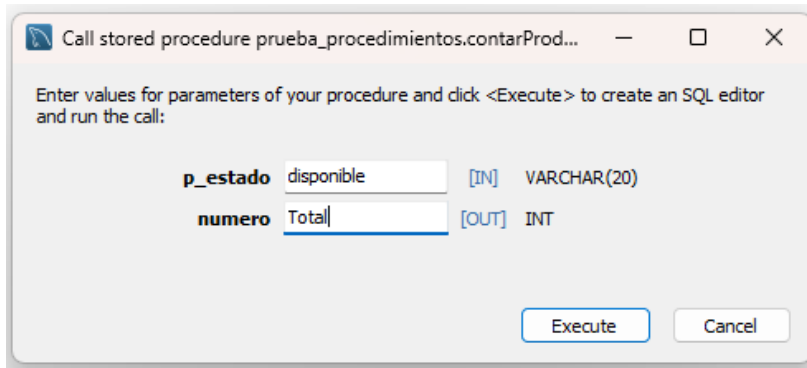
```
CREATE PROCEDURE `contarProductosEstado` (IN p_estado VARCHAR(20), OUT numero
INT)
BEGIN
    SELECT COUNT(id)
    INTO numero
    FROM producto
    WHERE estado = p_estado;
END
```

Luego, aplicar los cambios



Ejecutar Procedimiento Salida

Ejecutar desde el panel izquierdo y colocar los parámetros:



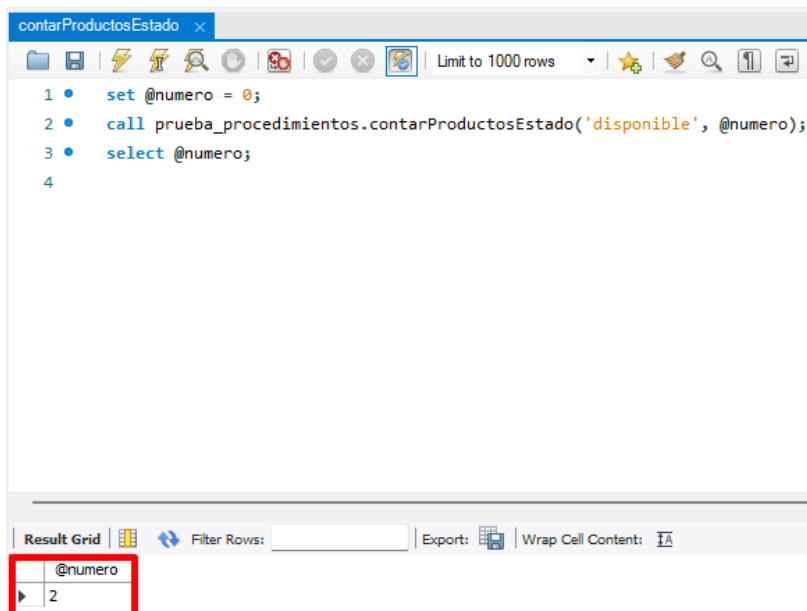
Call stored procedure prueba_procedimientos.contarProd...

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

p_estado disponible [IN] VARCHAR(20)

numero Total [OUT] INT

Execute Cancel



contarProductosEstado x

Limit to 1000 rows

```
1 • set @numero = 0;
2 • call prueba_procedimientos.contarProductosEstado('disponible', @numero);
3 • select @numero;
4
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

| | @numero |
|---|---------|
| ▶ | 2 |

Crear procedimiento Entrada/Salida

Crear un procedimiento llamado **venderProducto** que incremente una variable llamada “ganancia” cuando se vende un producto.

En el procedimiento se pasa el parámetro ganancia como INOUT para que mantenga su valor tras la ejecución del procedimiento. También se define el parámetro producto_id como IN para decirle al procedimiento el producto que se desea vender. Para hacerlo más sencillo, se entiende que el producto siempre está disponible.

El procedimiento recibe dos parámetros:

INOUT ganancia

IN producto_id

- **INOUT ganancia:** recibe la ganancia actual y devuelve la ganancia actualizada.
- **IN producto_id:** recibe el ID del producto que se desea vender.

Procedimiento

```
CREATE PROCEDURE `venderProducto` (INOUT ganancia FLOAT, IN producto_id INT)
BEGIN
    SELECT precio INTO @INCREMENTO
    FROM producto
    WHERE id = producto_id;
    SET ganancia = (ganancia + @incremento);
END
```

¿Cómo funciona?

1. Se recibe el **ID del producto** mediante producto_id.
2. Se busca su **precio** en la tabla producto.
3. El precio se guarda temporalmente en @incremento.
4. Se suma el precio a ganancia.
5. Como ganancia es INOUT, **el nuevo valor se conserva después de ejecutar el procedimiento.**

Por ejemplo:

```
SET @ganancia = 10000;
CALL venderProducto(@ganancia, 1);
SELECT @ganancia;
```

Si el producto 1 cuesta \$20.000, el resultado será:

30000

Nota: IN recibe información, mientras que INOUT **recibe un valor, lo modifica y devuelve el nuevo valor.**

Ejecutar procedimiento entrada/salida

Call stored procedure prueba_procedimientos.venderProd...

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

ganancia 2500 [INOUT] float
producto_id 1 [IN] int

Execute Cancel

```

1 • set @ganancia = 2500;
2 • call prueba_procedimientos.venderProducto(@ganancia, 1);
3 • select @ganancia;
4

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

| @ganancia |
|-----------|
| 22500 |

Eliminar procedimiento

Para eliminar el procedimiento solo es necesario utilizar la sentencia DROP seguida del nombre del procedimiento.

DROP PROCEDURE venderProducto;

Taller Procedimientos Almacenados

- Crear una nueva base de datos llamada biblioteca con codificación utf_8
- Crear una tabla llamada libros con la siguiente estructura:
 - codigo entero (2) Auto Incremental – PK (*)
 - titulo varchar (40)
 - autor varchar (30)
 - editorial varchar (20)
 - precio doble
 - stock entero (5)

Insertar valores

| Titulo | Autor | Editorial | Precio | Stock |
|-------------------------------------|--------------------------|--------------|---------|-------|
| Alicia en el país de las maravillas | Lewis Carroll | Planeta | 20,000 | 9 |
| Alicia en el país de las maravillas | Lewis Carroll | Mundial | 25,000 | 50 |
| Aprenda a Programar desde cero | Mauro Chojin | Siglo XX | 75,000 | 10 |
| Cien años de Soledad | Gabriel Garcia Marquez | Exclusivo | 155,000 | 25 |
| El Principito | Antoine de Saint-Exupéry | Panamericana | 29,000 | 75 |
| Uno | Richard Bach | Planeta | 25,000 | 65 |
| Los Secuestradores de Burros | Gerald Durrell | Siglo XX | 81,549 | 10 |

- Crear procedimientos almacenados que resuelvan las siguientes solicitudes.
 - Cantidad de libros donde el Stock sea menor o igual a 10
 - Mostrar los libros cuyo valor sea inferior a \$50,000
 - Venta de libros: cada vez que se venda un libro, se deberá descontar del stock.
 - Mostrar los libros por un autor específico.

Se debe crear un manual mostrando todos los pasos de la ejecución, subir el archivo SQL a la plataforma de AVACO en el enlace dispuesto para tal fin.